

Reducing Pipeline Bubbles in Message-Efficient State Machine Replication

Yiliang Wan¹, Inho Choi¹, Nitin Shivaraman², Akshaye Shenoi³,
Xiang Liu¹, Tao Luo², Haifeng Sun¹, Chaoyi Ruan^{1*}, Jialin Li¹

¹National University of Singapore

²IAIC, A*Star Singapore

³ETH Zurich

Abstract

Round-based pipelining in state machine replication (SMR) protocols reduces amortized bottleneck message complexity to $O(1)$, but waiting for network propagation and accommodating timing variation create pipeline bubbles that can erase this benefit. We present CHORA, a message-efficient SMR protocol co-designed with a network runtime to minimize these bubbles under common-case synchrony. CHORA introduces a new round construct that decouples propagation delay from round length. It then builds a carefully engineered runtime that minimizes variation in processing jitter. The co-designed protocol leverages strong synchrony to achieve $O(1)$ amortized bottleneck message complexity with high resource utilization, and maintains safety and liveness when timing predictability degrades. Evaluation shows that CHORA reaches 3.92 million operations per second with five RDMA replicas, achieving $2.87\times$ throughput of the state-of-the-art, while maintaining comparable latency. Even on this single-hop RDMA network that favors conventional rounds, CHORA reduces time lost to pipeline bubbles, leaving more CPU time for protocol processing. Across three to seven replicas, the CPU utilization rate for protocol processing increases by 211%–421% and throughput improves by $5.3\text{--}6.6\times$ over conventional rounds.

1 Introduction

State machine replication (SMR) [39] is the standard technique for building fault-tolerant services: replicas agree on a common sequence of client operations and execute them in the same order, preserving linearizability [19] even when some replicas fail. SMR underlies widely deployed systems such as Chubby [5], ZooKeeper [20], and Spanner [8]. Because replication sits on the critical path of these systems, the throughput of the SMR protocol often limits the capacity of the services built on it.

SMR protocols usually improve throughput by *pipelining*, overlapping multiple consensus instances to hide communication delay. Once enough proposals are in flight, however, throughput is bounded by the *bottleneck message complexity*: the number of messages the busiest replica must process per committed proposal. In conventional leader-based protocols [26, 35], the leader handles $O(N)$ messages per proposal,

where N is the replica count. Multi-leader protocols [32, 33] distribute proposal work across replicas, improving utilization and load balancing, but each proposer still handles $O(N)$ messages for its own proposals. *Aggregation* reduces this cost further by letting a single message carry work for many instances: a replica’s outgoing proposal can also carry a cumulative acknowledgement for proposals it has already received from other replicas. The effectiveness of aggregation, however, depends on transmission timing: responding immediately to each arrival aggregates little, while waiting for more progress adds latency and may stall on a delayed peer. Without predictable message arrival and processing times, it is difficult to coordinate this pipeline while keeping both waiting and message-processing overhead low.

Round-based pipelining makes replication *message-efficient* by coordinating aggregation to reduce the number of messages replicas send and process per committed proposal. In synchronous *rounds* [2, 30], each replica multicasts one message per round and processes that round’s peer messages before advancing. Every replica receives all $O(N)$ proposals of a round and can piggyback $O(N)$ acknowledgements on its next outgoing message. This reduces the amortized bottleneck message complexity to $O(1)$. The round length depends on the synchrony bound which includes the maximum propagation and processing delays. Our insight is that the approach trades the message bottleneck for timing: throughput now depends on how tightly rounds can be sized physically.

This approach poses two practical challenges. First, timing bounds can be violated in practice, so protocol correctness cannot depend on synchrony guarantees; this is why practical SMR protocols assume only partial synchrony [13, 26, 29, 35], where delays are bounded during sufficiently stable periods of execution. Second, even when a bound holds, conventional rounds waste much of the available processing capacity: a round must cover both network propagation and protocol processing, and because propagation dominates in typical deployments, replicas spend much of each round idle waiting for messages to arrive—a *propagation bubble*. Additional slack reserved for variation in delivery and processing times creates a *jitter bubble*. Measurements (§3) show that these bubbles can erase pipelining’s throughput benefit.

We present CHORA, a *message-efficient* SMR protocol that achieves *zero-bubble pipelining* in the common case through co-design with a network runtime. Zero-bubble execution

*Corresponding author.

requires timing that is tightly predictable rather than merely bounded; the runtime provides this *strong synchrony*. Because the two bubbles arise from different sources, one from the round’s structure and one from timing variation, the design consists of two components, one for each bubble.

First, the *CHORA protocol* (§5) makes correctness independent of synchrony and removes the propagation bubble. In the common case, each replica multicasts one *pulse* per round but processes available messages from earlier rounds, overlapping network propagation with useful work so the slowest link no longer dictates round length. Each pulse combines a new proposal with a cumulative acknowledgement for earlier proposals, achieving $O(1)$ amortized bottleneck message complexity; we call this common-case execution the *pulsing mode*. When timing predictability degrades, the same protocol state and commit rules drive a *responsive mode* that transmits protocol messages immediately, so safety and liveness never depend on synchrony.

Second, the *runtime* (§6) realizes strong synchrony in practice, so that the rounds the protocol defines logically remain short in physical time. It shrinks the jitter bubble by isolating round processing on a dedicated core, separating application handling and packet I/O through FIFO queues and kernel-bypass networking. Adaptive round boundaries (a minimum duration and a soft deadline with early close) let each round end as soon as its expected work completes, without budgeting every round for rare tail delays. A controller tunes these boundaries online and switches the group between pulsing and responsive execution when synchrony degrades.

Together, the two components remove both bubbles from common-case execution while retaining correctness under partial synchrony. We evaluate CHORA with RDMA and UDP transports on a local cluster and on AWS EC2. With five RDMA replicas, CHORA reaches 3.92 Mops/s, $2.87 \times \mu$ [1], and its advantage persists in intra-AZ and cross-AZ cloud deployments. An ablation shows that propagation-decoupled rounds reduce time lost to pipeline bubbles, leaving more CPU time for protocol processing. Across three to seven replicas, the CPU utilization rate for protocol processing increases by 211%–421% and throughput improves by 5.3–6.6 $\times$ over conventional rounds. Finally, pulsing generalizes beyond CHORA: adapting the pulsing mode to Derecho [22, 23] and Hermes [24] improves their throughput by 41.6% and 28.1–41.8%, respectively.

2 Background

State machine replication (SMR) [39] replicates a deterministic state machine across a set of servers. Replicas agree on a common sequence of operations and execute them in the same order, providing linearizability [19] even when a subset of replicas fails. In this work, we assume all replicas

follow the protocol, and can only fail by crashing. SMR protocols have been widely deployed to provide strong fault tolerance guarantees in distributed systems [4, 5, 8, 20].

Replicated logs organize operations in practical SMR protocols [26, 29, 32, 35]. Replicas coordinate to assign proposals to log slots, and a proposal becomes committed once the replicas reach agreement on its position in the log. A correct SMR protocol provides two key guarantees: *safety* prevents conflicting proposals from being committed at the same log slot, while *liveness* ensures that submitted proposals eventually get committed.

Synchrony affects liveness and performance; safety must hold without timing guarantees. Consensus cannot guarantee progress in a fully asynchronous system with failures [15], so practical protocols assume partial synchrony [13, 26, 29, 35, 43], under which message delivery and processing are eventually bounded, and rely on it for liveness. Synchrony also shapes performance: message delay bounds commit latency, and protocols with stronger timing guarantees, such as synchronized clocks or bounded delays, exploit them to order operations with less coordination [5, 8, 12, 14, 17, 18].

Proposal and acknowledgement form the two stages of a leader-based commit [26, 29, 35]. In the *proposal* stage, the leader assigns a client command to the next log slot and sends the proposal to all replicas. In the *acknowledgement* stage, each replica records the proposal and sends an acknowledgement back to the leader, which commits the proposal once it has received acknowledgements from a quorum. Each stage involves a network traversal and message processing at both ends, so a proposal takes at least one round trip to commit.

SMR pipelining overlaps work across log slots to improve resource utilization and throughput. Committing one proposal at a time would leave replicas idle for most of the round trip. The leader starts the next slot’s proposal stage before earlier slots finish their acknowledgement stage, keeping multiple consensus instances in flight. At any moment, the leader is sending proposals for new slots while receiving acknowledgements for earlier ones, and each replica alternates between processing incoming proposals and sending acknowledgements. Multi-phase BFT protocols pipeline in the same way, overlapping protocol stages across successive proposals or blocks [10, 46].

3 Motivation and Challenges

SMR protocols pipeline consensus instances to overlap their communication and processing. This section examines what limits such a pipeline and how synchrony helps. Responsive pipelining is bounded by its bottleneck message complexity, and round-based pipelining removes this bound by aggregating proposals and acknowledgements across instances (§3.1). Rounds, however, leave two kinds of idle time, or *bubbles*: a *propagation bubble* from placing network propagation inside

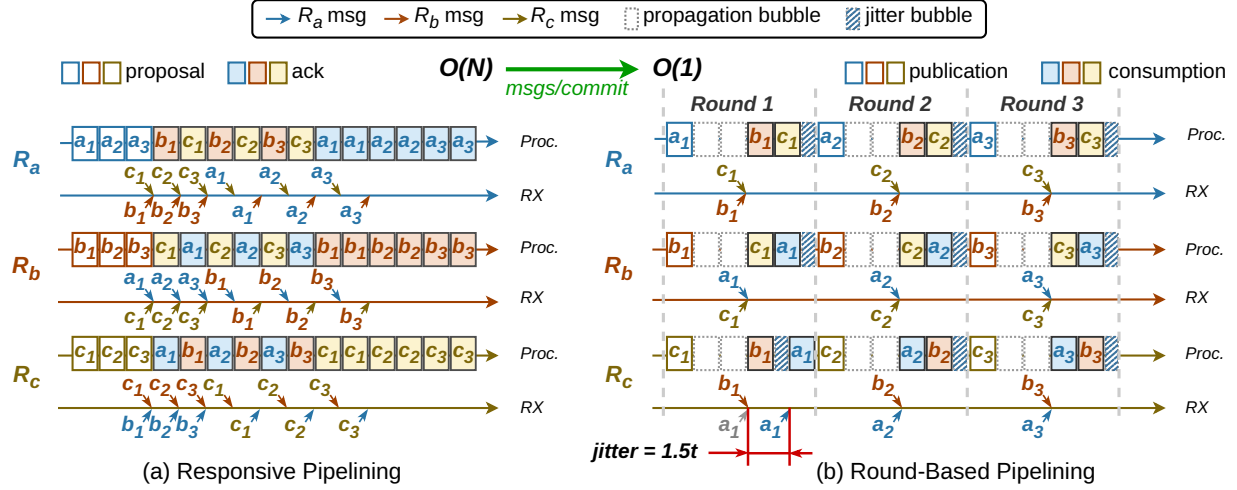


Figure 1. Round-based pipelining reduces amortized bottleneck message complexity from $O(N)$ in (a) to $O(1)$ in (b). Each block costs t , and propagation takes $\delta = 2t$.

the round barrier, and a *jitter bubble* from variation in message delivery and processing times (§3.2). We then analyze how each bubble bounds pipeline performance (§3.3).

3.1 From Responsive to Round-Based Pipelining

Responsive pipelining and its bottleneck. As discussed in §2, practical SMR protocols pipeline multiple consensus instances to hide communication latency and improve resource utilization. Conventional pipelining follows a *responsive* execution model: replicas process protocol messages as they arrive and immediately perform the corresponding protocol actions. Different consensus instances thus progress independently, without coordinating their transmissions through common round boundaries. Responsive pipelining hides end-to-end communication latency but does not reduce the protocol work required for each proposal. Once the pipeline is saturated, throughput is determined by its *bottleneck message complexity*, i.e., the number of protocol messages processed by the busiest replica per committed proposal.

For leader-based protocols [29, 35, 43], the bottleneck is the leader, which processes an acknowledgement from every replica for each commit and therefore handles $O(N)$ messages per proposal, where N is the replica count. Multi-leader (or equivalently, leaderless) SMR protocols [32, 33, 41, 42] address this bottleneck by letting all replicas propose. As Figure 1(a) illustrates, this utilizes the otherwise idle compute cycles on non-leader replicas and distributes the proposer workload across the cluster. However, each commit still requires its proposer to collect acknowledgements from a quorum, so the bottleneck message complexity remains $O(N)$. Moreover, the lack of a single designated leader introduces coordination overhead, such as blocking on slow leaders in Mencius [32] and conflict resolution for dependent operations in EPaxos [33].

Rounds amortize bottleneck message complexity. Both designs have $O(N)$ bottleneck message complexity because they treat concurrent consensus instances independently. If replicas instead coordinate their transmissions, a single protocol message can carry progress for multiple instances. In particular, a replica can combine a new proposal with a cumulative acknowledgement for all proposals it has received into one multicast. A conventional synchronous round [2, 30] has two phases. In the *publication phase*, a replica constructs and multicasts a message; in the *consumption phase*, it receives and processes peer messages sent in that round. Round-based execution of this kind is common in DAG-based BFT protocols, where each replica broadcasts one block per round that references the blocks it received in earlier rounds [10, 25, 40]. As Figure 1(b) shows, with N replicas each multicasting once per round, each replica processes $O(N)$ messages while the pipeline advances N proposals, yielding $O(1)$ amortized bottleneck message complexity.

3.2 Two Bubbles in Round-Based Pipelining

Rounds, however, are not free. Replicas now advance in lock-step, and any time a replica spends waiting within a round is idle time, or a *bubble*, in the pipeline. Figure 1(b) marks two kinds: a *propagation bubble* (dotted) that comes from the round structure itself, and a *jitter bubble* (hatched) that comes from timing variation.

Propagation bubble. A conventional round requires its consumption phase to process peer messages published in the same round. In Figure 1(b), a replica multicasts and processes its own proposal at the start of a round, but its peers' messages arrive only after the propagation delay, so it idles until they do and then processes them. If δ is the mean one-way propagation delay and t is the mean per-message processing time, the round must therefore cover approximately $\delta + Nt$

even under ideal timing. The idle δ is the propagation bubble. It grows with the longest propagation delay between any pair of replicas, since the round cannot end before the slowest message has arrived and been processed.

The bubble stems from the same-round requirement, not from coordination itself. Our key observation is that coordinating *transmission* does not require consuming messages from the same round. We therefore introduce *propagation-decoupled rounds*: publication still multicasts once per round, while consumption processes peer messages from earlier rounds. In Figure 3, each replica processes its own proposal together with the messages its peers sent in the previous round, so propagation overlaps with useful processing and a round holds only the N processing blocks. Messages on different links may even lag by different numbers of rounds without making the propagation delay of the slowest link part of every round (Figure 5), so the ideal round length reduces from approximately $\delta + Nt$ to Nt . Propagation still contributes to end-to-end commit latency, but it no longer limits the steady-state pipeline rate.

Jitter bubble. Decoupling removes the mean propagation delay from the round, but a second bubble remains. Replicas must still cross each round boundary together, so the round length must cover not only the mean processing time but also its variation. In Figure 1(b), a peer’s message may arrive up to $1.5t$ late, so every round reserves that much slack, and a replica that finishes early idles until the boundary before issuing its next pipeline step. Conversely, if the round is too short, some replicas fail to complete their processing within the round and must defer unfinished work to later rounds. Round-based pipelining thus faces a trade-off: long rounds leave replicas idle, while short rounds increase the likelihood of missed round boundaries. Either way, the bubble can negate the throughput benefit of reduced bottleneck message complexity and lead to worse latency. Unlike the propagation bubble, its size is set by how unpredictable message delivery and processing are, not by their mean.

This is where stronger common-case synchrony becomes useful. Suppose that the variance of message delivery and processing latencies is low, and their latency distributions are known to all replicas. With this timing information, replicas can choose a tight round bound such that each replica completes its expected processing within the round with high probability. When the variance is low, the round length only needs to cover the mean processing time plus a small buffer for the tail, so most of the round consists of useful work and the jitter bubble is small. This property does not require equal propagation delays across links, since propagation-decoupled rounds allow messages from different links to span different numbers of rounds. What matters is low variation in message delivery and protocol processing. When such timing predictability is unavailable, replicas can instead progress responsively based on message arrivals, preserving progress without relying on tight round coordination.

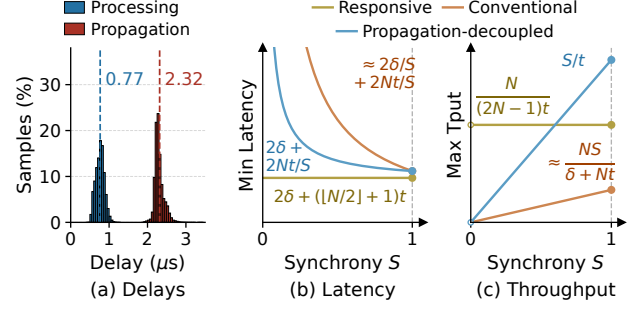


Figure 2. Round delays and analytical bounds. (a) Measured round processing and one-way propagation; dashed lines mark means. (b–c) Optimal latency and maximum throughput versus synchrony S ($\delta \gg t$).

3.3 Pipeline Performance Bottleneck Analysis

How much each bubble costs depends on the timing of the deployment. The propagation bubble depends on how propagation delay compares with processing delay, and the jitter bubble depends on how predictable delivery and processing are. We analyze each in turn, first with measurements of propagation and processing delays, then with an analytical model as functions of a synchrony coefficient.

Propagation delay versus processing delay. A conventional round of length $\delta + Nt$ spends δ waiting, so the propagation bubble occupies a fraction $\delta/(\delta + Nt)$ of every round. Figure 2(a) measures both terms on our one-hop three-replica RDMA testbed at saturation. Mean one-way propagation is $2.32 \mu\text{s}$, three times the $0.77 \mu\text{s}$ a replica spends processing a round’s messages, so the propagation bubble takes about three quarters of a conventional round even after message aggregation. The two means approximate the measured round lengths with fixed deadlines and early exit: $\delta + Nt = 3.09 \mu\text{s}$ against $3.45 \mu\text{s}$ for conventional rounds, and $Nt = 0.77 \mu\text{s}$ against $0.83 \mu\text{s}$ for propagation-decoupled rounds. The gap in each case is the jitter bubble, which we quantify next.

Quantifying synchrony. The jitter bubble is the slack a round reserves for timing variation, e.g., the 0.5 to $1.1 \mu\text{s}$ spread of per-round processing in Figure 2(a). We quantify it with the *synchrony coefficient* $S \in (0, 1]$: the fraction of a round spent on useful processing once the round is bounded to cover that variation, so $1 - S$ implies the jitter bubble. $S = 1$ means no slack is needed, and a smaller S means a looser round bound. S^x denotes the coefficient for a bound at the x -th percentile. Appendix A gives the formal definition of synchrony coefficient S .

Figure 2(b–c) plots the resulting bounds against S , modeling all slack with a common factor $1/S$. A conventional round lasts about $(\delta + Nt)/S$, bounding throughput at $NS/(\delta + Nt)$; a propagation-decoupled round lasts about Nt/S , bounding throughput at S/t , which reaches the processing limit $1/t$ as $S \rightarrow 1$. Both improve with tighter synchrony, but only decoupled rounds escape δ . A responsive multi-leader pipeline,

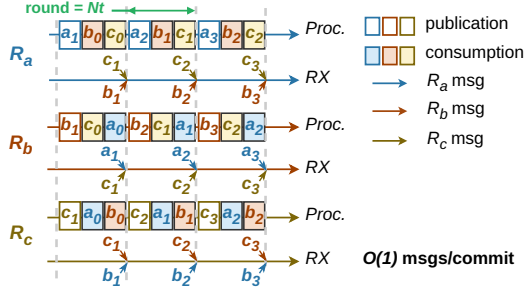


Figure 3. Zero-bubble pipelining. Propagation-decoupled rounds overlap processing (t per block) with propagation ($\delta = 2t$ one-way), shortening ideal rounds to Nt while retaining $O(1)$ amortized bottleneck message complexity.

in contrast, is bounded at $N/((2N - 1)t)$ regardless of S , since each replica also performs follower work for the other proposers. The model assumes a uniform per-message cost t ; actual costs vary, but every message incurs deserialization, validation, and state updates, so bottleneck message complexity remains a fair proxy for processing work.

Figure 2(b) shows the other side of the trade-off. Every design pays two propagation legs, 2δ , for a proposal to reach the replicas and their acknowledgements to return. The responsive baseline adds only path processing, about $(\lfloor N/2 \rfloor + 1)t$, whereas round-based pipelines add two round crossings: $2\delta/S + 2Nt/S$ for conventional rounds, which stretch each propagation leg, and $2\delta + 2Nt/S$ for decoupled rounds. With the delays of Figure 2(a), that is about $1 \mu s$ on top of $4.6 \mu s$ of propagation, so rounds cost little latency once decoupled.

These bounds set the targets for CHORA: remove the δ term with propagation-decoupled rounds, and push S toward one with a strong-synchrony runtime. The next section outlines how CHORA realizes both.

4 CHORA Overview

To address the above bubbles, we present CHORA, an SMR protocol co-designed with a strong-synchrony network runtime that realizes *zero-bubble* SMR pipelining. CHORA preserves the message efficiency of round-based aggregation while eliminating idle gaps between successive pipeline stages. CHORA addresses the propagation bubble and jitter bubble through two techniques.

- **Propagation-decoupled pipeline (§5).** CHORA removes the same-round dependency between publication and consumption to eliminate the propagation bubble. Unlike a conventional round, in which replicas transmit and process messages from the same round, a propagation-decoupled round processes messages transmitted in previous rounds. Messages for future pipeline stages propagate concurrently with the processing of messages that have already arrived. Once the pipeline is filled, network propagation is removed from the critical path of each round,

CHORA Replica Local State

- **log** - replication log
- **cmds** - buffered client request commands
- **view** - current view number
- **role** - current role (initiator, candidate, or follower)
- **voted-for** - candidate that this replica voted for
- **voted-by** - set of replicas that voted for this replica
- **view-base** - first log slot of the current view
- **next-propose** - next log slot to propose
- **last-append** - log slot before the first empty slot
- **last-ack** - last acknowledged slot in the current view
- **last-commit** - last committed log slot
- **acked** - acknowledgement indices, one per replica

Figure 4. Replica state in CHORA.

allowing replicas to advance proposal, acknowledgment, and commitment stages continuously. We design an SMR protocol that puts the propagation-decoupled pipeline into practice. The CHORA protocol uses a high-performance *pulsing mode* under strong synchrony and a *responsive fallback* when this timing property does not hold.

- **Strong-synchrony runtime (§6).** CHORA runtime builds an execution substrate that provides strong synchrony for short, predictable rounds and minimizes the jitter bubble. Conventional partial synchrony assumes only an eventual finite delay bound and suffices for protocol liveness. Strong synchrony further requires communication and processing delays to have low variance and a tight probabilistic bound. It is a common-case performance property rather than an additional safety assumption. CHORA isolates timing-sensitive protocol operations from application and transport activity, then adapts flexible round boundaries to observed timing and protocol progress. It monitors clock and pipeline health and switches to responsive execution when strong synchrony degrades.

The two techniques address complementary dimensions of the pipeline. Propagation-decoupled rounds overlap *mean* network propagation with protocol processing, while strong synchrony minimizes the residual waiting required for *tail* latency variation. Together, they produce the common-case execution shown in Figure 3, in which proposal transmission, acknowledgment, and commitment advance across rounds without pipeline bubbles.

5 CHORA Protocol

To achieve zero-bubble pipelining, we design the CHORA protocol based on the propagation-decoupled pipeline. We first overview the protocol in §5.1. We then describe the primary pulsing mode in §5.2 and its responsive fallback in §5.3. Finally, we discuss how to recover from different failure types in §5.4. Appendix B gives the complete message specification, transition rules, and safety proof.

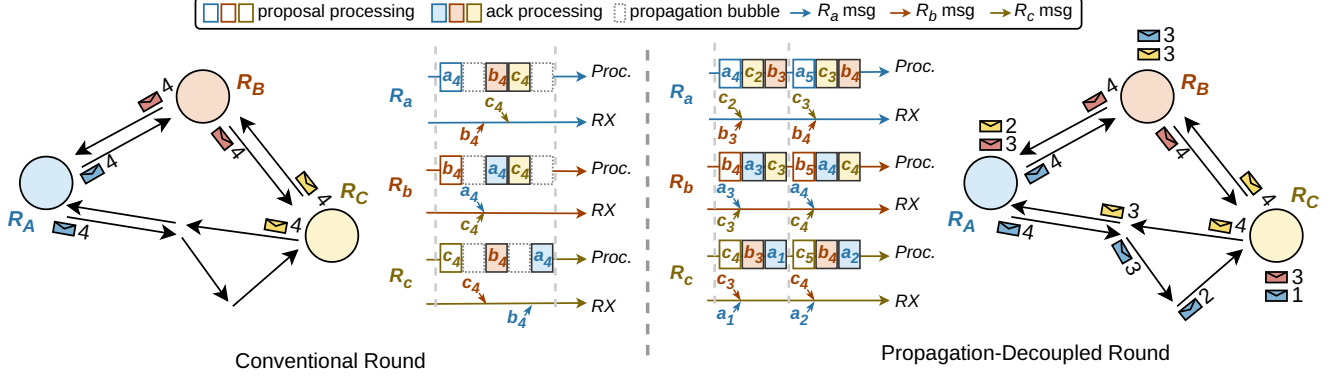


Figure 5. Propagation-decoupled rounds process messages from different earlier rounds, keeping the propagation delay out. Envelope labels give publication rounds (e.g., $\text{✉}3$ denotes a pulse published in round 3). Each black arrow represents the unit time of propagation delay ($\Delta\delta = t$, where t is per-message processing time).

5.1 Protocol Overview

Protocol model. CHORA replicates a deterministic state machine across $N = 2f + 1$ replicas and tolerates up to f crash failures. Replicas follow the protocol but may crash or become unreachable. Byzantine behavior is outside of our scope. The protocol guarantees linearizable execution and preserves correctness under partial synchrony. When favorable strong synchrony exists, CHORA leverages it to achieve improved performance.

Views and slot assignment. The protocol proceeds through monotonically numbered *views*. In each view, every replica is assigned a subset of the log slots and may propose commands only for its assigned slots. CHORA assigns slots in a round-robin manner, making each replica the leader for a disjoint subset of the log. A view change reassigns the log space when the proposer set changes.

Replica state. Figure 4 summarizes the local state maintained by each replica. Note that *last-append* is determined by the first empty slot in the log, which is updated implicitly whenever a log entry is added or removed. A replica acknowledges slot s only after receiving *all* proposals through s , i.e., the acknowledgment in CHORA is *cumulative*.

Dual-mode transmission. The key challenge is to exploit strong synchrony for zero-bubble pipelining without making correctness depend on this stronger timing property. CHORA thus separates replication semantics from transmission scheduling. It uses *pulsing mode* to realize the propagation-decoupled pipeline under strong synchrony and falls back to *responsive mode* under partial synchrony. Both modes operate on the same protocol state and safety rules, so switching modes does not require protocol reconfiguration.

5.2 Pulsing Mode

Pulsing mode is the primary execution mode of CHORA. It maps the two-stage SMR pipeline onto propagation-decoupled

rounds, which process messages from earlier rounds while current messages propagate, eliminating the propagation bubble from the critical path. This mode operates over the strong-synchrony network provided by CHORA runtime (see §6), where message delays are bounded and relatively stable. Replicas can therefore advance through short, time-driven rounds without waiting excessively for delayed messages, minimizing the jitter bubble. Together, propagation-decoupled rounds and the strong-synchrony runtime enable zero-bubble pipelining in the common case.

In pulsing mode, replicas advance through coordinated rounds, i.e., each active replica multicasts at most one message per round, called a *pulse*. Each round’s publication and consumption phases advance the proposal and acknowledgment stages of different log slots.

Figure 5 contrasts these round models with three replicas. On the left, each replica processes peer messages published in the current round, so the round must accommodate the slowest current-round propagation path, producing the dashed propagation bubbles. On the right, a replica processes messages published in different earlier rounds according to when they become available. The envelope labels indicate their publication rounds: messages processed together need not have been transmitted in the same round. A slower link therefore increases the round lag of messages arriving over that link but does not extend every round. Meanwhile, messages from the current round propagate for future processing while replicas perform proposal or acknowledgment processing on messages that have already arrived.

We detail the protocol operations of each phase in pulsing mode as follows.

Publication phase. A client sends its request to any proposer; the default assignment lets every replica propose. The replica buffers the request in *cmds* until its next proposal

opportunity. Replica r constructs a pulse containing a proposal for *next-propose* and its cumulative acknowledgment *last-ack*. After sending the pulse, r advances *next-propose* to its next assigned slot. A pulse carries information from different pipeline instances. It introduces a new proposal while reporting cumulative acknowledgment progress for proposals processed in earlier rounds. This aggregation removes the need for a separate acknowledgment transmission for every proposal in the common case.

Consumption phase. When a replica consumes a valid proposal from replica r , it records the command at the proposed slot and monotonically updates *acked* [r] with the attached cumulative acknowledgment. If the proposal fills the first hole in the local log, the replica advances *last-append* and *last-ack* across the newly completed prefix. A proposal received beyond an earlier hole may be buffered but cannot advance the acknowledged prefix. Let $q = f + 1$ be the quorum size. The q -th largest entry in *acked* identifies the longest log prefix acknowledged by a quorum. The replica advances *last-commit* to the smaller of this quorum index and its own acknowledged prefix. It executes newly committed commands in order and replies to the clients whose requests it originally received. The replica includes the updated *last-ack* in a subsequent pulse publication phase, allowing the acknowledgment stage for these proposals to overlap with the proposal stage for newer slots. After the pipeline is filled, every round disseminates new proposals while processing, acknowledging, and potentially committing proposals from earlier rounds. When all R participating replicas are proposers, a normal round uses R multicast messages to advance up to R proposals. The protocol therefore retains $O(1)$ amortized bottleneck message complexity while overlapping network propagation with useful protocol processing.

5.3 Responsive Mode

When strong synchrony cannot be sustained, CHORA falls back to responsive mode. Responsive mode applies the same slot assignment, log, acknowledgement, and commitment rules as pulsing mode, but sends protocol information immediately rather than waiting for the next pulse.

When a client request arrives, its receiving replica immediately proposes the request in its next assigned slot. A replica receiving the proposal advances its acknowledged prefix when possible and sends an explicit PROPOSE-ACK. Negative acknowledgements and recovery messages are likewise transmitted as soon as they are generated. Responsive mode sacrifices the message aggregation of pulsing mode but preserves progress under partial synchrony.

Because the two modes differ only in transmission scheduling, switching modes does not require a view change or log reconfiguration. An acknowledgement piggybacked on a pulse and an explicit PROPOSE-ACK provide the same replication evidence and update the same protocol state.

5.4 Failure Recovery

Failures appear as stalled advancement of the committed prefix. CHORA first attempts lightweight recovery within the current view and performs a view change when it suspects that a responsible proposer is unavailable.

Detecting stalled progress. Each proposer maintains a coordination timer for its locally proposed but uncommitted commands. When the timer expires, the replica selects its earliest uncommitted proposal at slot s and checks the interval from *last-ack* + 1 through s . If *last-ack* < s , its local log is missing earlier proposals and it requests those slots. Otherwise, other replicas may have missed its proposal at s , so it retransmits the proposal. Responsive mode sends the resulting messages immediately, whereas pulsing mode defers them to the next pulse. The proposer resets the timer after each timeout action and whenever a local proposal commits.

In-view recovery. A replica missing slot s multicasts a PROPOSE-NACK identifying the missing slot and its view. A replica retaining the proposal in its acknowledged prefix can respond with a PROPOSE-RECOVER carrying the proposal, allowing one multicast to repair the same log hole at multiple replicas. In pulsing mode, a proposer replacing a fresh proposal with recovery accounts for its next assigned slot with a no-op and advances its proposal pointer. In responsive mode, an idle proposer fills its unused assigned slots on request; recovering an existing proposal does not itself consume a fresh slot. A SKIP message represents a range of no-ops compactly. These mechanisms restore progress without changing the current view.

Replica failure. If a proposer crashes or becomes partitioned, its assigned slots may remain permanently empty and cannot be repaired within the current view. A candidate then collects votes from a quorum using an at-least-as-up-to-date log comparison. Once elected as the new initiator, it proposes a VIEW-INIT entry that excludes suspected failed replicas and defines a new slot assignment. Followers reconcile their logs with the initiator before VIEW-INIT commits, after which replicas install the new assignment and resume normal operation. The full view-change procedure and its safety argument appear in Appendix B.

6 CHORA Runtime

The CHORA runtime implements the propagation-decoupled rounds of §5 in physical time. It shrinks the jitter bubble, which is the residual timing variation in processing and message arrival. As a result, each round lasts only as long as its expected work requires. Successive rounds then pack back-to-back. This yields zero-bubble common-case execution. Strong synchrony provides the timing predictability this requires. To this end, the runtime isolates timing-sensitive protocol operations on a critical path (§6.1), adapts round boundaries to observed timing (§6.2), and falls back to responsive execution when short coordinated rounds become

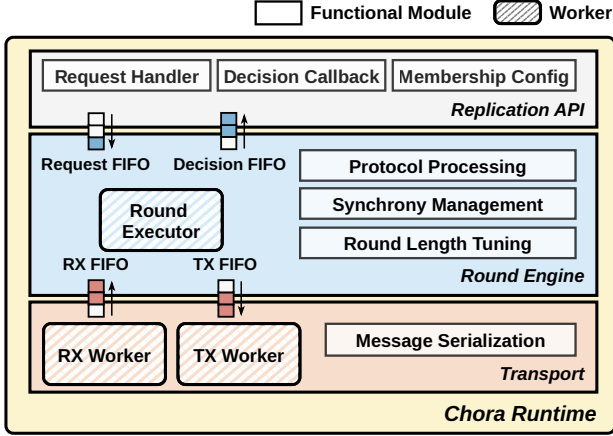


Figure 6. CHORA runtime architecture.

ineffective (§6.3). As shown in Figure 6, it consists of a replication API, a round engine, and a transport layer connected by FIFO queues; one replica’s round engine also acts as the *controller*, scheduling execution across replicas—a role the view initiator (§5) can host.

6.1 Isolating Round Operations

Round critical path. Realizing short physical rounds requires the work that determines round progress to have predictable cost. The runtime therefore separates operations that determine round progress from operations that can proceed asynchronously. The critical path contains only the protocol operations required to complete the publication and consumption phases in §5.2. For publication, these operations select buffered requests, construct the next proposal, attach the cumulative acknowledgement, and hand off the resulting pulse. For consumption, they apply available peer pulses to the log and acknowledgement state and advance the committed prefix. In contrast, client admission, application execution, and packet I/O do not directly advance the round and therefore remain outside its critical path.

Critical-path isolation. The round engine executes the critical-path operations in a dedicated *round executor*, isolating them from application, transport, and execution-resource interference. First, the replication API performs client admission and application delivery outside the executor. *Request* and *decision FIFOs* connect the two, allowing the executor to consume admitted requests and return committed decisions without synchronizing with the application. Second, dedicated *RX* and *TX workers* perform packet I/O and exchange messages with the executor through separate FIFOs. The executor hence updates protocol state without blocking on network I/O. Finally, the executor can run on a dedicated core, while the transport workers can use kernel-bypass I/O to reduce resource contention and packet-processing variance [3, 16, 36, 47]. Together, these isolation boundaries make the cost of critical-path operations more predictable.

Arrival decoupling. Predictable processing alone is insufficient if each round must wait for a particular network arrival. The RX FIFO thus also decouples round execution from individual packet timing. In the propagation-decoupled pipeline, the consumption phase may process pulses published in different earlier rounds. That is, it does not need to wait for all pulses from one designated round. The FIFO can absorb short arrival bursts and gaps while the executor consumes pulses that are already available. Consequently, stable propagation delay changes the number of rounds for which a pulse remains in flight rather than the round length. Only arrival variation that drains the FIFO is exposed to the critical path.

6.2 Adaptive Round Boundaries

Flexible round boundaries. Strong synchrony makes communication and processing delays tightly concentrated, but not identical across rounds. A fixed round must either cover rare tail delays or risk expiring during a protocol handler. We therefore use coordinated rounds with flexible termination within a bounded timing window. Specifically, CHORA runtime sets a *hard minimum round length* τ and a *soft maximum deadline* T . The minimum limits the pulse publication rate. The maximum prevents the executor from waiting indefinitely for peer pulses, but is soft because the executor checks it only between atomic protocol operations.

Let the receive quota be one pulse from each of the other $N - 1$ replicas. A round closes in one of four cases:

- The quota is complete before τ : the executor waits until τ and then closes the round.
- The quota becomes complete between τ and T : the executor closes the round immediately.
- The quota is still incomplete at T while the executor is polling: the executor closes the round at the deadline.
- A protocol operation finishes after T : the executor closes the round immediately after that operation completes.

These rules allow common-case rounds to finish as soon as the expected protocol operations are complete, while containing tail events without requiring every round to budget for them. The controller adapts both limits, keeping τ large enough to respect the configured pulse-rate limit.

Optimization objective. Choosing the round limits trades pipeline *efficiency* against *effectiveness*. Longer rounds can complete more useful work before advancing, but may spend more time waiting; shorter rounds pack boundaries more tightly, but may defer work to later rounds. We formalize this trade-off as follows. For a continuously backlogged stream of W commands, let an ideal pipeline commit the commands in r_0 rounds of length t_0 . For round limits $c = (\tau, T)$ with $\tau \leq T$, let $r(c)$ be the required number of rounds and $\bar{t}(c)$ their mean actual length. We define

$$\alpha(c) = \frac{t_0}{\bar{t}(c)}, \quad \beta(c) = \frac{r_0}{r(c)},$$

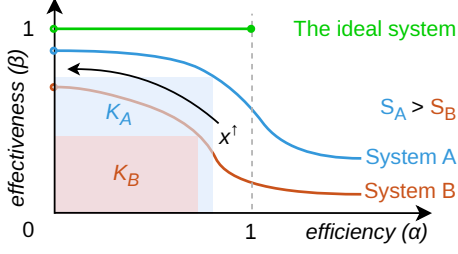


Figure 7. Pipeline efficiency–effectiveness trade-off (S is the synchrony coefficient).

where α is pipeline efficiency (how tightly rounds are packed in time) and β is pipeline effectiveness (useful progress per round). Their product gives throughput normalized to the ideal rate Tput_0 :

$$\frac{\text{Tput}(c)}{\text{Tput}_0} = \frac{W/[r(c)\bar{t}(c)]}{W/(r_0 t_0)} = \alpha(c)\beta(c).$$

The peak normalized throughput over configurations c is $\kappa = \max_c \alpha(c)\beta(c)$. As shown in Figure 7, a more predictable execution sustains higher effectiveness at a given efficiency and therefore achieves a larger κ . This connects strong synchrony directly to pipeline performance: lower timing variation allows shorter rounds without sacrificing useful progress.

Estimating the objective. The controller evaluates each candidate configuration against one fixed ideal reference. For a given protocol, workload, replica count, and hardware, offline profiling measures the active processing time of a fully productive round with all required inputs already available. Its mean provides the reference round length t_0 , while the protocol’s ideal progress determines r_0 for the same amount of work. For each tested configuration c , the runtime then measures two quantities: the mean actual round length $\bar{t}(c)$, including waiting and overruns, and the achieved committed progress per round, which determines $r(c)$. Plugging these measurements into the definitions above yields estimated efficiency $\hat{\alpha}(c)$ and effectiveness $\hat{\beta}(c)$. Their product estimates normalized throughput, and the highest value among tested configurations gives $\hat{\kappa} = \max_c \hat{\alpha}(c)\hat{\beta}(c)$.

Applying an update. The controller explores nearby configurations through bounded changes to τ and T and distributes each candidate to all replicas. It retains a candidate only when its estimated improvement persists across measurement windows; otherwise, it restores the previous limits. When samples or load are insufficient to evaluate a candidate reliably, it leaves the current configuration unchanged.

6.3 Synchrony-Aware Mode Switching

CHORA runtime uses pulsing mode while strong synchrony sustains short, coordinated rounds and falls back to responsive mode when these timing conditions degrade. To determine when to switch, each replica reports clock health (synchronization status and error bound) together with pipeline

health (persistent overruns, quota misses, and messages per committed operation). The controller evaluates these signals over observation windows. Sparse control pulses maintain measurements during low traffic.

The controller manages mode transitions across replicas without modifying protocol state. When the monitored signals indicate sustained degradation, it disseminates a decision to switch to responsive mode, carrying a monotonic sequence number. On receipt, a replica stops publishing timed pulses and switches to responsive processing for messages. No view change or state conversion is needed because pulsing and responsive execution use the same log and acknowledgment state and differ only in transmission scheduling (§5.3). After a longer healthy window, the controller schedules the group’s return to pulsing with a conservative initial deadline. Adaptation resumes once the round pipeline stabilizes. Different entry and return thresholds prevent repeated switching near the synchrony boundary.

7 Generalizing the Pulsing Mode

Pulsing is not specific to CHORA. It applies to any replication system whose protocol progress can be summarized cumulatively and published independently of when that progress is consumed. We adapt such a system in two steps: identify the *cumulative progress* whose latest value summarizes work across multiple operations, then combine it with new work in propagation-decoupled pulses, so replicas can publish later pulses without waiting for the current one to be consumed. This reorganizes how protocol work is communicated and scheduled while preserving the system’s original replication semantics and completion rules. We demonstrate the generalization on Derecho (§7.1) and Hermes (§7.2), two systems with different replication models.

7.1 Derecho

Derecho [22, 23] provides ordered multicast by separating data dissemination from agreement on delivery. Its protocol already maintains cumulative progress, so only the second step is needed: publishing this existing progress in propagation-decoupled pulses without changing how Derecho orders or delivers messages.

Cumulative progress. Derecho maintains shared protocol state in a *shared state table* (SST). In its small-message path, a sender publishes the availability of new payloads, while replicas advance cumulative SST counters as those payloads are received and processed. Because later counter values subsume earlier ones, the latest SST state summarizes progress across many messages and serves directly as Derecho’s cumulative progress.

Propagation-decoupled pulsing. Each pulse publishes the latest payload availability together with the sender’s current cumulative progress. Payloads are made available before the corresponding progress is published, preserving Derecho’s

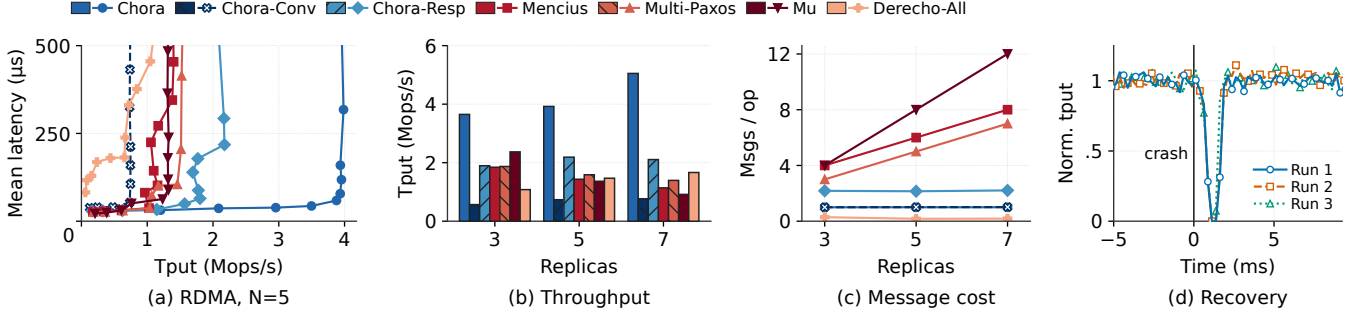


Figure 8. CHORA sustains higher throughput as replicas increase and recovers after a crash. Panel (c) counts TX messages plus RX processing steps triggered by peer updates per committed command at the busiest replica. Each multicast counts once; Mu and Derecho count RDMA writes per destination. Panel (d) normalizes each run to its mean throughput before the crash.

existing ordering between data and metadata. After sending a pulse, a replica continues processing already available work and may publish later pulses without waiting for responses to the current one.

To keep rounds short and predictable, we execute Derecho’s native SST processing in local rounds on a dedicated thread, isolated from communication work. Protocol work need not align one-to-one with pulses: Derecho retains its native batching and delivery mechanisms, while each pulse publishes the latest cumulative progress.

7.2 Hermes

Hermes [24] provides linearizable key-value replication using invalidations and per-key logical timestamps. Unlike Derecho, Hermes tracks progress through acknowledgments and validations associated with individual writes, so adaptation needs the first step as well: constructing a cumulative representation of that progress.

Cumulative progress. A Hermes write begins when a replica broadcasts an invalidation containing the key, timestamp, and new value; the writer collects acknowledgments from every other replica before broadcasting a validation. Logical timestamps resolve concurrent writes to the same key, while early value propagation allows another replica to replay an interrupted write. To make this per-write progress cumulative, we assign sequence numbers to invalidations within each origin worker’s stream. For every origin, a replica maintains a *processed prefix*: the largest sequence number up to which all invalidations from that origin have passed through the replica’s native invalidation handler. Because the prefix advances monotonically, one value summarizes processing of all preceding invalidations from that origin; for each replica, these prefixes form a vector with one entry per origin. In the same way, each origin maintains a *completed prefix* covering writes for which it has collected all acknowledgments.

Propagation-decoupled pulsing. Each pulse carries newly generated invalidations with their sequence numbers, the replica’s vector of processed prefixes, and the writer’s completed prefix. For each origin, the minimum processed prefix

reported by all replicas identifies the prefix of writes that every replica has processed. This cumulative evidence replaces the exchange of individual acknowledgments for the covered writes, while the writer still runs Hermes’s native acknowledgment logic, including its key, timestamp, and replay checks, on each newly covered write. The writer’s completed prefix plays the same role for validation: it identifies the writes ready for validation without individual validation messages, while a follower runs Hermes’s native validation handler, including the per-key timestamp check, on covered writes up to its own processed prefix.

As with Derecho, each Hermes worker processes its protocol work in local rounds: at the start of a round, it seals its prefixes and newly generated work and broadcasts them as a pulse, then consumes available invalidations and progress until the round ends.

8 Evaluation

We evaluate whether CHORA reduces pipeline bubbles while retaining the low amortized bottleneck message complexity of round-based pipelining. We compare end-to-end results (§8.2), isolate the mechanisms (§8.3), and test its generality (§8.4). Except when studying round length, we tune CHORA’s round limits following the objective in §6.2.

8.1 Implementation and Experimental Setup

Implementation. We implement CHORA as a C library and use DPDK 25.11.0 for its runtime (23.11 on AWS). The runtime supports RDMA (RoCEv2) and UDP. We run RDMA with unreliable datagrams (UD) for hardware multicast.

Testbeds. The local cluster uses Xeon Gold 6226R/6326 CPUs (2.90 GHz) and ConnectX-5 NICs. We use four isolated cores per replica. Cloud experiments use AWS EC2 m7i.2xlarge instances with spread placement and UDP, either within one availability zone (intra-AZ) or across zones (cross-AZ).

Baselines. Our baselines isolate protocol choices from the effects of pulsing. Multi-Paxos and Mencius provide single-leader and rotating multi-leader baselines. CHORA-Resp and

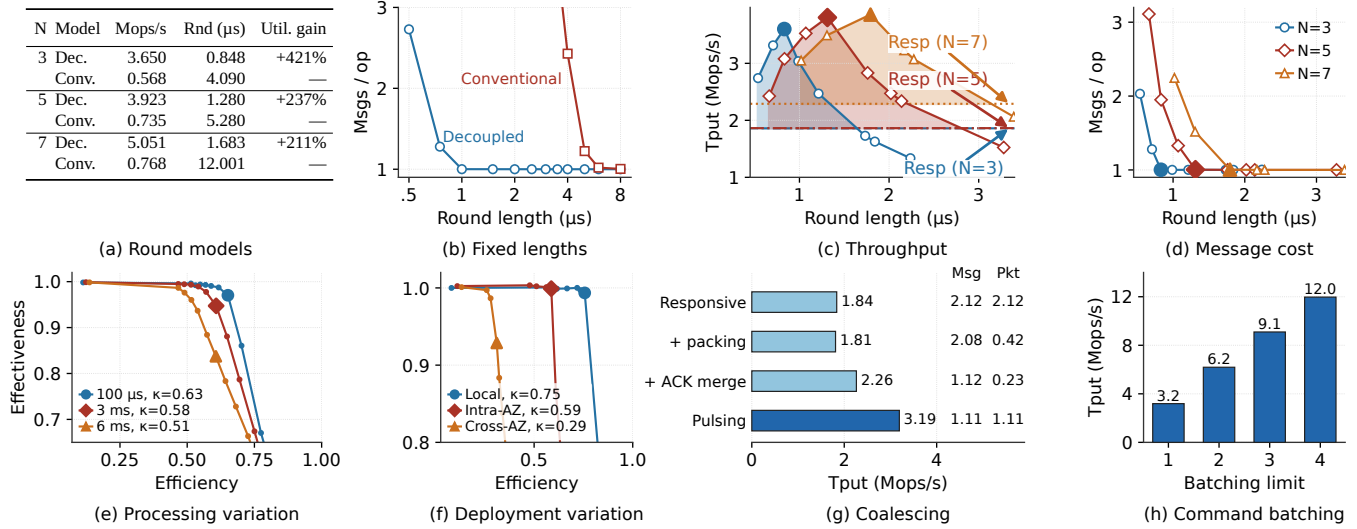


Figure 9. Reducing pipeline bubbles improves efficiency while retaining effectiveness; command batching adds further gains. In (a), Dec./Conv. denote decoupled/conventional rounds; Util. gain is the relative utilization increase over conventional rounds. Horizontal lines in (c) show responsive throughput; large markers in (e,f) maximize κ . Message and packet metrics sum what all replicas send per committed command, counting each multicast once.

CHORA-Conv run our protocol with responsive execution and conventional rounds, respectively. All four use CHORA runtime. CHORA-Conv accepts only current-round pulses. We also use the optimized open-source RDMA implementations of Mu [1] and Derecho [22, 23], enabling all Derecho senders (Derecho-All).

Workload. SMR clients spread evenly across proposers send 64-byte commands over kernel UDP on a separate network fabric. Proposals carry at most one command except in Derecho, which retains native batching. §8.3 separately studies command batching in CHORA with its round policy fixed.

8.2 End-to-End Performance

On local RDMA, CHORA combines higher throughput with low-load latency close to responsive execution, sustains its advantage as replicas increase, and recovers after a crash.

Latency and throughput with RDMA. With five replicas, CHORA leads at 3.92 Mops/s: 1.79 $\times$ CHORA-Resp and 2.87 $\times$ Mu (Figure 8b). At low load, CHORA and CHORA-Resp respond in 29.4 and 29.2 μ s, respectively (Figure 8a). Short round crossings add little waiting, while propagation remains part of commit latency (§3.3).

Scalability. CHORA sustains its throughput advantage as the replica count grows from three to seven (Figure 8b–c). With seven replicas, it exceeds the strongest baseline, CHORA-Resp, by 2.40 $\times$. Round-based aggregation keeps amortized bottleneck message complexity near one message per committed command; Multi-Paxos and Mu handle more messages as replicas increase.

The propagation bubble still limits CHORA-Conv despite its low message cost, leaving processing capacity idle. Derecho’s native batching reduces message cost below one; §8.4 evaluates coordinating its processing passes with pulsing.

Crash recovery. After a replica crash, CHORA resumes processing in about 2 ms and returns near its previous throughput (Figure 8d). We stop one replica’s DPDK application near peak throughput in each of three runs. Heartbeat timers trigger a view change to exclude the failed replica; the survivors then shorten their round timeouts.

8.3 Pipelining Mechanisms and Timing

We test how decoupling and synchrony reduce pipeline bubbles to improve efficiency while retaining effectiveness, then compare pulsing with coalescing and batching.

Propagation-decoupled rounds. Propagation-decoupled rounds remove the propagation bubble by overlapping propagation with processing of pulses from earlier rounds. Both round models share the protocol, runtime, and throughput-feedback controller, with fixed membership and offered load and no batching or coalescing. Only decoupled rounds use early exit. Reducing time lost to pipeline bubbles leaves more CPU time for protocol processing. Across three to seven replicas, the CPU utilization rate for protocol processing increases by 211%–421% over conventional rounds (Figure 9a). Throughput improves by 5.3–6.6 $\times$.

Without tuning or early exit, decoupled rounds retain low message cost at shorter lengths (Figure 9b). Decoupled rounds send about one pulse per operation from 1 μ s upward; conventional rounds need about 6 μ s to approach that cost. Shorter conventional rounds discard late pulses and

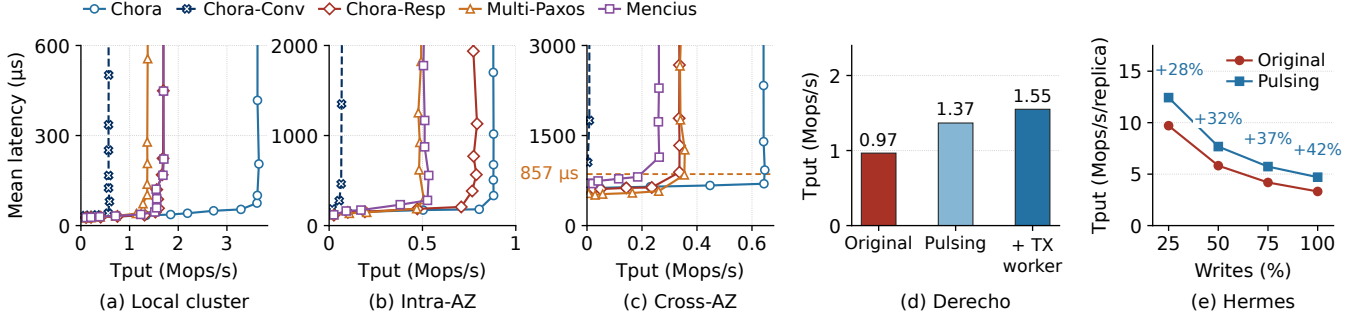


Figure 10. Pulsing improves throughput across deployments (a–c) and systems (d,e) with three replicas. Panels (d,e) use fixed membership and best tested originals at saturation. The dashed line in (c) marks the lowest measured mean latency for Multi-Paxos clients farthest from the leader.

retransmit proposals, making no progress at $2.5 \mu\text{s}$ and below. Decoupled rounds also incur extra pulses when too short.

Round length. Throughput peaks when rounds are just long enough for processing, balancing pipeline efficiency and effectiveness (Figure 9c–d). Short rounds leave operations unfinished, lowering effectiveness and increasing messages per commit. Long rounds leave capacity idle, lowering efficiency. Pulsing exceeds responsive throughput throughout the shaded ranges, so gains extend beyond one deadline setting. CHORA uses this trade-off to select deadlines (§6.2).

Synchrony. Stronger synchrony sustains higher pipeline effectiveness at a given efficiency by reducing the jitter bubble (Figure 9e). We inject uniformly sampled receive-thread delays with ranges of $100 \mu\text{s}$, 3 ms , and 6 ms in a local three-replica deployment. Deployment measurements show the same trade-off. Peak normalized throughput falls from 0.75 locally to 0.59 within an AZ and 0.29 across AZs (Figure 9f).

Relation to coalescing and batching. Pulsing adds gains beyond coalescing and complements command batching. With three RDMA replicas, each panel uses 64 -byte proposals for coalescing and 256 -byte proposals for batching. Coalescing reduces traffic but does not match pulsing’s throughput (Figure 9g). Packing reduces packets; cumulative ACK merging also removes superseded acknowledgments without changing proposal processing. Both use a $10 \mu\text{s}$ interval, selected for ACK merging from a 0.5 – $10 \mu\text{s}$ sweep. ACK merging raises responsive throughput from 1.84 to 2.26 Mops/s; pulsing reaches 3.19 Mops/s, 41% higher despite similar logical traffic. Traffic totals sum across senders. Command batching raises throughput without changing the round policy (Figure 9h). With up to four commands per proposal, throughput rises from 3.19 to 11.97 Mops/s. The executor takes queued commands without waiting to fill batches; sparse load still produces single-command proposals.

8.4 Generality

Pulsing also improves throughput across deployments and when applied to Derecho and Hermes (§7).

Cloud deployments. Pulsing retains its throughput advantage in the cloud despite longer and more variable network delays. We compare local, intra-AZ, and cross-AZ deployments over UDP, excluding the RDMA-based Mu and Derecho baselines (Figure 10a–c). We keep all systems within cloud packet-rate allowances using longer CHORA rounds and smaller responsive windows where needed. Within one AZ, CHORA reaches 0.88 Mops/s versus 0.79 for CHORA-Resp. Across AZs, it reaches 0.65 Mops/s versus 0.36 for Multi-Paxos, the strongest baseline there. CHORA-Conv reaches only 0.07 Mops/s within an AZ and 0.009 Mops/s across AZs, because its lock-step rounds must cover the cloud’s long propagation delays as well as protocol processing. Accepting commands at every replica lets clients use a nearby replica in wider deployments without routing through a fixed leader. CHORA’s cross-AZ low-load latency is about $620 \mu\text{s}$ across zones; Multi-Paxos ranges from $402 \mu\text{s}$ in its leader’s zone to $857 \mu\text{s}$ in the farthest zone (dashed line).

Derecho Adaption. Pulsing improves Derecho’s throughput by coordinating its existing processing passes. Throughput rises from 0.966 to 1.367 Mops/s, a 41.6% gain without extra workers (Figure 10d). The metric counts globally ordered deliveries across senders. A TX worker adds 13.4% , reaching 1.550 Mops/s. We retain native batching and compare against the best tested original from a configuration sweep. Pulsing thus complements Derecho’s native batching.

Hermes Adaption. Pulsing raises Hermes throughput by 28.1 – 41.8% , with greater gains as writes increase (Figure 10e). This trend is consistent with sharing acknowledgment and validation work across writes. Throughput counts completed requests per replica, including local reads. Both variants use two workers per replica without communication offload, accessing one million keys uniformly with 31 -byte values. We give both implementations equal tuning effort and compare their best tested configurations at each write fraction.

9 Discussion and Related Work

Network models. Most SMR protocols assume partial synchrony [13] for liveness [6, 15, 26, 29, 35]. The timed asynchronous model [9] supports reasoning about timing without full synchrony. Speculative Paxos [38] provides mostly-ordered multicast through SDN. Network sequencers reduce coordination in NOPaxos [28], Eris [27], and Hydra [7].

Datacenter replication. Datacenter replication systems optimize communication and data movement, and recent work also exploits datacenter synchrony. DARE [37], APUS [44], and Mu [1] optimize replication and consensus with RDMA. Hermes [24], Derecho [22], and Spindle [23] exploit concurrency, pipelining, and batching. The Synchronous Data Center [45] argues for fully synchronous data centers, and Watchmaker [34] and SSR [31] build SMR on synchronized time slots or rounds. These systems primarily optimize communication and data movement; CHORA instead exploits predictable communication and processing timing to coordinate SMR execution more efficiently.

Hardware acceleration. CHORA can benefit from hardware acceleration, including embedding time synchronization in protocol messages via NIC hardware timestamps, reducing processing tail latency through careful message handling, and offloading consensus to programmable hardware such as SDN switches [11] or FPGAs [21, 31].

10 Conclusion

We present CHORA, a message-efficient SMR protocol co-designed with a network runtime to realize zero-bubble pipelining. First, its propagation-decoupled pipeline processes earlier-round messages while publishing current proposals, eliminating the propagation bubble without sacrificing message efficiency. Second, the runtime isolates timing-sensitive operations and adapts round boundaries, minimizing the jitter bubble under strong synchrony. When strong synchrony degrades, CHORA falls back to responsive execution while preserving correctness and progress under ordinary synchrony.

References

- [1] Marcos K. Aguilera, Naama Ben-David, Rachid Guerraoui, Virendra J. Marathe, Athanasios Xygiak, and Igor Zablotchi. 2020. Microsecond Consensus for Microsecond Applications. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 599–616.
- [2] Amotz Bar-Noy, Xiaotie Deng, Juan A. Garay, and Tiko Kameda. 1991. Optimal amortized distributed consensus. In *Proceedings of the 5th International Workshop on Distributed Algorithms (WDAG)*. Springer, 95–107.
- [3] Adam Belay, George Prekas, Ana Klimovic, Samuel Grossman, Christos Kozyrakis, and Edouard Bugnion. 2014. IX: A Protected Dataplane Operating System for High Throughput and Low Latency. In *Proceedings of the 11th USENIX Conference on Operating Systems Design and Implementation (OSDI '14)*. USENIX Association, 49–65.
- [4] William J. Bolosky, Dexter Bradshaw, Randolph B. Haagens, Norbert P. Kusters, and Peng Li. 2011. Paxos replicated state machines as the basis of a high-performance data store. In *Proceedings of the 8th USENIX Conference on Networked Systems Design and Implementation (NSDI '11)*. USENIX Association, 141–154.
- [5] Mike Burrows. 2006. The Chubby lock service for loosely-coupled distributed systems. In *Proceedings of the 7th symposium on Operating systems design and implementation*. 335–350.
- [6] Miguel Castro and Barbara Liskov. 1999. Practical Byzantine Fault Tolerance. In *Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI '99)*. USENIX Association, Co-sponsored by IEEE TCOS and ACM SIGOPS.
- [7] Inho Choi, Ellis Michael, Yunfan Li, Dan R. K. Ports, and Jialin Li. 2023. Hydra: Serialization-Free Network Ordering for Strongly Consistent Distributed Applications. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, 293–320. <https://www.usenix.org/conference/nsdi23/presentation/choi>
- [8] James C Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, Jeffrey John Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, et al. 2013. Spanner: Google's globally distributed database. *ACM Transactions on Computer Systems (TOCS)* 31, 3 (2013), 1–22.
- [9] Flaviu Cristian and Christof Fetzer. 1999. The Timed Asynchronous Distributed System Model. *IEEE Transactions on Parallel and Distributed Systems* 10, 6 (1999), 642–657.
- [10] George Danezis, Lefteris Kokoris-Kogias, Alberto Sonnino, and Alexander Spiegelman. 2022. Narwhal and Tusk: a DAG-based mempool and efficient BFT consensus. In *Proceedings of the Seventeenth European Conference on Computer Systems (EuroSys '22)*. Association for Computing Machinery, 34–50. doi:10.1145/3492321.3519594
- [11] Huynh Tu Dang, Daniele Sciascia, Marco Canini, Fernando Pedone, and Robert Soulé. 2015. NetPaxos: Consensus at Network Speed. In *Proceedings of the 1st ACM SIGCOMM Symposium on Software Defined Networking Research*. 1–7.
- [12] Jiaqing Du, Daniele Sciascia, Sameh Elnikety, Willy Zwaenepoel, and Fernando Pedone. 2014. Clock-RSM: Low-latency inter-datacenter state machine replication using loosely synchronized physical clocks. In *2014 44th Annual IEEE/IFIP International Conference on Dependable Systems and Networks*. IEEE, 343–354.
- [13] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer. 1988. Consensus in the presence of partial synchrony. *Journal of the ACM (JACM)* 35, 2 (1988), 288–323.
- [14] Vitor Enes, Carlos Baquero, Alexey Gotsman, and Pierre Sutra. 2021. Efficient replication via timestamp stability. In *Proceedings of the Sixteenth European Conference on Computer Systems (EuroSys '21)*. ACM, 178–193. doi:10.1145/3447786.3456236
- [15] Michael J. Fischer, Nancy A. Lynch, and Michael S. Paterson. 1985. Impossibility of Distributed Consensus with One Faulty Process. *J. ACM* 32, 2 (April 1985), 374–382. doi:10.1145/3149.214121
- [16] Linux Foundation. 2024. Data Plane Development Kit. <https://www.dpdk.org/>.
- [17] Jinkun Geng, Anirudh Sivaraman, Balaji Prabhakar, and Mendel Rosenblum. 2022. Nezha: Deployable and High-Performance Consensus Using Synchronized Clocks. *Proceedings of the VLDB Endowment* 16, 4 (Dec. 2022), 629–642. doi:10.14778/3574245.3574250
- [18] Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung. 2003. The Google File System. In *Proceedings of the Nineteenth ACM Symposium on Operating Systems Principles (SOSP '03)*. Association for Computing Machinery, 29–43. doi:10.1145/945445.945450
- [19] Maurice P Herlihy and Jeannette M Wing. 1990. Linearizability: A correctness condition for concurrent objects. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 12, 3 (1990), 463–492.
- [20] Patrick Hunt, Mahadev Konar, Flavio P Junqueira, and Benjamin Reed. 2010. ZooKeeper: Wait-free Coordination for Internet-scale Systems. In *2010 USENIX Annual Technical Conference (USENIX ATC 10)*.

- [21] Zsolt István, David Sidler, Gustavo Alonso, and Marko Vukolic. 2016. Consensus in a box: Inexpensive coordination in hardware. In *13th USENIX Symposium on Networked Systems Design and Implementation (NSDI 16)*. 425–438.
- [22] Sagar Jha, Jonathan Behrens, Theo Gkountouvas, Mae Milano, Weijia Song, Edward Tremel, Robbert Van Renesse, Sydney Zink, and Ken Birman. 2019. Derecho: Fast State Machine Replication for Cloud Services. *ACM Transactions on Computer Systems (TOCS)* 36, 2 (2019), 1–49.
- [23] Sagar Jha, Lorenzo Rosa, and Ken Birman. 2022. Spindle: Techniques for Optimizing Atomic Multicast on RDMA. In *2022 IEEE 42nd International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 1085–1097. doi:10.1109/ICDCS54860.2022.00108
- [24] Antonios Katsarakis, Vasileios Gavrielatos, M.R. Siavash Katebzadeh, Arpit Joshi, Aleksandar Dragojevic, Boris Grot, and Vijay Nagarajan. 2020. Hermes: A Fast, Fault-Tolerant and Linearizable Replication Protocol. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 201–217.
- [25] Idit Keidar, Lefteris Kokoris-Kogias, Oded Naor, and Alexander Spiegelman. 2021. All You Need is DAG. In *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing*. 165–175.
- [26] Leslie Lamport. 2001. Paxos Made Simple. *ACM SIGACT News* 32, 4 (Dec. 2001), 51–58.
- [27] Jialin Li, Ellis Michael, and Dan R. K. Ports. 2017. Eris: Coordination-Free Consistent Transactions Using In-Network Concurrency Control. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP '17)*. Association for Computing Machinery, 104–120. doi:10.1145/3132747.3132751
- [28] Jialin Li, Ellis Michael, Naveen Kr Sharma, Adriana Szekeres, and Dan RK Ports. 2016. Just Say NO to Paxos Overhead: Replacing Consensus with Network Ordering. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. 467–483.
- [29] Barbara Liskov and James Cowling. 2012. *Viewstamped Replication Revisited*. Technical Report MIT-CSAIL-TR-2012-021. MIT.
- [30] Nancy A Lynch. 1996. *Distributed algorithms*. Elsevier.
- [31] Yuke Ma, Paolo Costa, and Yiting Xia. 2026. SSR: Synchronous State Machine Replication with a Timing-Aligned Split Architecture. In *Proceedings of the 17th ACM SIGOPS Asia-Pacific Workshop on Systems*. 67–73. doi:10.1145/3838177.3841728
- [32] Yanhua Mao, Flavio P. Junqueira, and Keith Marzullo. 2008. Mencius: building efficient replicated state machines for WANs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation (OSDI'08)*. USENIX Association, 369–384.
- [33] Iulian Moraru, David G. Andersen, and Michael Kaminsky. 2013. There is more consensus in Egalitarian parliaments. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles (SOSP '13)*. Association for Computing Machinery, 358–372. doi:10.1145/2517349.2517350
- [34] Karan Newatia, Robert Gifford, Qingjie Lu, Andreas Haeberlen, and Linh Thi Xuan Phan. 2026. Running Distributed Systems like Clockwork. In *1st New Ideas in Networked Systems (NINeS 2026) (OASICS, Vol. 139)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 26:1–26:31. doi:10.4230/OASICS.NINeS.2026.26
- [35] Diego Ongaro and John Ousterhout. 2014. In Search of an Understandable Consensus Algorithm. In *Proceedings of the USENIX Conference on USENIX Annual Technical Conference (USENIX ATC '14)*. USENIX Association, 305–319.
- [36] Simon Peter, Jialin Li, Irene Zhang, Dan R. K. Ports, Doug Woos, Arvind Krishnamurthy, Thomas Anderson, and Timothy Roscoe. 2014. Arrakis: The Operating System is the Control Plane. In *Proceedings of the 11th USENIX Conference on Operating Systems Design and Implementation (OSDI '14)*. USENIX Association, 1–16.
- [37] Marius Poke and Torsten Hoefer. 2015. DARE: High-Performance State Machine Replication on RDMA Networks. In *Proceedings of the 24th International Symposium on High-Performance Parallel and Distributed Computing (HPDC)*. ACM, 107–118.
- [38] Dan RK Ports, Jialin Li, Vincent Liu, Naveen Kr Sharma, and Arvind Krishnamurthy. 2015. Designing distributed systems using approximate synchrony in data center networks. In *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*. 43–57.
- [39] Fred B. Schneider. 1990. Implementing fault-tolerant services using the state machine approach: a tutorial. *Comput. Surveys* 22, 4 (dec 1990), 299–319. doi:10.1145/98163.98167
- [40] Alexander Spiegelman, Neil Giridharan, Alberto Sonnino, and Lefteris Kokoris-Kogias. 2022. Bullshark: DAG BFT Protocols Made Practical. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 2705–2718.
- [41] Chrysoula Stathakopoulou, Matej Pavlovic, and Marko Vukolić. 2022. State machine replication scalability made simple. In *Proceedings of the Seventeenth European Conference on Computer Systems*. 17–33.
- [42] Sarah Tollman, Seo Jin Park, and John Ousterhout. 2021. EPaxos Revisited. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*. 613–632.
- [43] Robbert Van Renesse and Deniz Altinbuken. 2015. Paxos Made Moderately Complex. *Comput. Surveys* 47, 3 (Feb. 2015), 1–36. doi:10.1145/2673577
- [44] Cheng Wang, Jianyu Jiang, Xusheng Chen, Ning Yi, and Heming Cui. 2017. APUS: Fast and Scalable Paxos on RDMA. In *Proceedings of the 2017 Symposium on Cloud Computing (SoCC)*. ACM, 94–107.
- [45] Tian Yang, Robert Gifford, Andreas Haeberlen, and Linh Thi Xuan Phan. 2019. The Synchronous Data Center. In *Proceedings of the Workshop on Hot Topics in Operating Systems (HotOS '19)*. ACM, 142–148. doi:10.1145/3317550.3321442
- [46] Maofan Yin, Dahlia Malkhi, Michael K Reiter, Guy Golan Gueta, and Ittai Abraham. 2019. HotStuff: BFT consensus with linearity and responsiveness. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing*. 347–356.
- [47] Irene Zhang, Amanda Raybuck, Pratyush Patel, Kirk Olynky, Jacob Nelson, Omar S. Navarro Leija, Ashlie Martinez, Jing Liu, Anna Kornfeld Simpson, Sujay Jayakar, Pedro Henrique Penna, Max Demoulin, Piali Choudhury, and Anirudh Badam. 2021. The Demikernel Data-path OS Architecture for Microsecond-Scale Datacenter Systems. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP '21)*. Association for Computing Machinery, 195–211. doi:10.1145/3477132.3483569

A Modeling Synchrony for SMR Pipelines

Coordinating the SMR pipeline requires a timing model for replica communication and processing. We define synchrony metrics that relate variation in these delays to the round length. Using these metrics, we analyze the performance of conventional round-based protocol constructs. We then introduce a new network round model to further improve the processing efficiency. Next, we discuss trade-offs of the CHORA round model to better understand its performance. Lastly, we describe a new network-level API based on the new round model. A new replication protocol that builds on this model and API is introduced in §5.

A.1 Quantifying System Synchrony

In §1, we introduced the synchrony coefficient $S \in (0, 1]$ as a simplified metric for network synchrony. We now give a formal definition that accounts for the percentile of the delay distribution. Similar to prior models, our synchrony definition centers around message delays in a networked system. However, we further refine the model to define a *degree* of synchrony, instead of a fixed delay bound. Suppose the message delay is represented by a random variable d . We define a x^{th} synchrony coefficient as:

$$\tilde{S}^x = \frac{\mu(d)}{p_{x^{th}}(d)} \quad (1)$$

where $p_{x^{th}}(Z)$ and $\mu(Z)$ denote the x^{th} percentile and the expected value of random variable Z , respectively.

This coefficient quantifies the synchrony degree. $p_{x^{th}}(d) = \frac{1}{\tilde{S}^x} \mu(d)$ specifies a time bound for receivers to complete processing a message after transmission, with an expectation of $x\%$. $\frac{1}{\tilde{S}^x} - 1$ represents the required relative time buffer to tolerate the tail delays. If the network is perfectly synchronous, $\mu(d) = p_{x^{th}}(d)$, which results in $\tilde{S}^x = 1$.

Nodes in prior fully synchronous protocols move in lock-step rounds. This enables synchronous and coordinated behavior across the network. In such a round, each node multicasts a message in the beginning, and then receives and processes the messages from other nodes. We define $\Delta \tilde{T}^x$ to be the time bound that a node can finish all processing for a round with an expectation of $x\%$. In the normal case, a node can receive and process more than one message in each round. Therefore, $\Delta p_{x^{th}}(d)$ is the lower bound of $\Delta \tilde{T}^x$. Assuming that the system can complete a workload of ΔW_x on average in each round with such an expectation, then the system throughput upper bound can be described as:

$$Tput^x = \frac{\Delta W^x}{\Delta \tilde{T}^x} \leq \tilde{S}^x \frac{\Delta W^x}{\mu(d)} \quad (2)$$

From the equation, we can see that a more synchronous system (with a larger S_x) provides better performance.

A.2 Propagation-Decoupled Rounds

Let's look at $\Delta \tilde{T}_x$ from another perspective. We further decompose the message delay d into a network induced propagation delay d_{prop} and a delay introduced by the application as a processing delay d_{proc} . Note that d_{prop} includes queuing delay, transmission delay, processing delay in the network stack, and propagation delay in the physical medium. With d_{prop} and d_{proc} , we have:

$$\begin{aligned} \Delta \tilde{T}_x &\geq p_{x^{th}}(d) \approx p_{x^{th}}(d_{prop}) + p_{x^{th}}(d_{proc}) \\ &= \mu(d_{prop}) + [p_{x^{th}}(d_{prop}) - \mu(d_{prop})] + \\ &\quad \mu(d_{proc}) + [p_{x^{th}}(d_{proc}) - \mu(d_{proc})] \end{aligned} \quad (3)$$

The equation indicates that $\Delta \tilde{T}_x$ is bounded by both the expected and the tail of both propagation delay and processing delay. With this observation, we introduce *CHORA rounds* to decrease its time bound (ΔT_x) for better performance.

Similar to a conventional synchronous round, in a CHORA round, a node also first multicasts a message and then receives and processes messages from other nodes. However, a CHORA round doesn't require nodes to process messages from the same round. This relaxation removes the expectation of propagation delay from the lower bound of $\Delta \tilde{T}_x$. In other words, for a CHORA round, we have:

$$\begin{aligned} \Delta T_x &\geq [p_{x^{th}}(d_{prop}) - \mu(d_{prop})] + \\ &\quad \mu(d_{proc}) + [p_{x^{th}}(d_{proc}) - \mu(d_{proc})] \end{aligned} \quad (4)$$

Figure 5 shows an example of a system operating on CHORA rounds. Each message is attached with a number that denotes the round when it is sent. In the next round ($i + 1$), node B will process node A's message from round i , while node C will process node A's message from round $i - 3$. Our new round structure allows the round length to be *decoupled* from the longest propagation delay (node A to C).

In practice, processing delays are often much lower than propagation delays on the critical path. For instance, on our local cluster one-way propagation averages $2.32 \mu s$, while a replica spends $0.77 \mu s$ on average processing a round's messages. With decoupled rounds, five replicas run rounds of about $1 \mu s$ and reach $5.3\times$ the throughput of conventional rounds.

With the new round definition and Equation 4, we rectify Equation 1 as:

$$S^x = \frac{\mu(d_{proc})}{p_{x^{th}}(d_{prop}) - \mu(d_{prop}) + p_{x^{th}}(d_{proc})} \quad (5)$$

Similar to Equation 2, we have the following formula that describes the relationship between S^x and the system's performance:

$$Tput^x = \frac{\Delta W^x}{\Delta T^x} \leq S^x \frac{\Delta W^x}{\mu(d_{proc})} \quad (6)$$

The equation shows that a more synchronous system, which has a higher S^x , is able to provide better performance. For a perfectly synchronous system where tail delays equal average delays, we have $P_{x^{th}}(d_{prop}) = \mu(d_{prop})$ and $P_{x^{th}}(d_{proc}) = \mu(d_{proc})$. In this case, the throughput upper bound is $\frac{\Delta W^x}{\mu(d_{proc})}$. This upper bound denotes a maximum processing utility rate of 100%.

A.3 Pipeline Performance

The throughput bound in Equation 6 trades work completed per round against round length. §6.2 develops this trade-off as pipeline efficiency and effectiveness and uses their product to motivate runtime deadline tuning. For the strict periodic rounds modeled here, actual and configured round lengths are equal. The runtime formulation also accounts for early exit and processing that crosses a soft deadline.

B Detailed CHORA Protocol

This appendix specifies the messages and state transitions underlying the protocol in §5. Figure 4 in the main paper lists the replica state. It covers normal operation, recovery, and view changes; §B.9 gives the safety argument. The message descriptions give logical fields and actions, independent of how a transport packs them into packets.

B.1 State, Views, and Slot Ownership

CHORA replicates a log across $N = 2f + 1$ replicas, with quorum size $q = f + 1$. Replicas follow the protocol and may crash; progress requires an eventual period of ordinary synchrony and a reachable quorum. Strong synchrony enables pulsing but is not a safety assumption. Each log entry contains a command, its slot, and the view in which its proposer created it. A command may contain one or more client requests or a protocol operation such as VIEW-INIT or NO-OP. Slot 0 contains an initial null command with view 0.

Prefix indices. All indices below are inclusive: *last-append* is the slot immediately before the first hole in the log, and *last-commit* is the last locally committed slot. The index *last-ack* marks the prefix the replica can acknowledge in its current view. During normal operation, it follows *last-append*; during a view change, a replica must first reconcile its uncommitted prefix with the new initiator. Thus, *last-commit* $\leq$ *last-ack* $\leq$ *last-append*. The array *acked* stores the largest acknowledgment received from each replica in the current view, including the local replica's own acknowledgment.

Slot ownership. A committed VIEW-INIT establishes the proposer set and slot assignment for its view. The slot *view-base* contains this command; ordinary proposals start at *view-base* + 1. For R proposers with offsets $i \in \{1, \dots, R\}$, the round-robin assignment gives proposer i slots *view-base* + $i + nR$ for nonnegative integers n . A proposer creates commands only for its assigned slots and never creates two different commands for the same slot in the same view. Its *next-propose*

identifies its first unused assigned slot; advancing this pointer selects the next slot in its assignment. A replica with no assigned slots still receives proposals and contributes acknowledgment evidence.

Message admission. Every replica message identifies its sender and current view. A replica ignores messages from older views and applies the transition rules in §B.8 before processing messages that establish a newer view. Normal processing requires an established slot assignment; retransmissions preserve the original slot, command, and command view. Duplicate messages do not create a second log entry or decrease an acknowledgment index. A replica never replaces an existing command with a different command in the same view.

B.2 Messages

The following fields describe the logical protocol messages. A recovery message carries the original proposal's view as part of its command, separately from the message's current view. An implementation may combine several logical records in one pulse.

REQUEST and REPLY A client sends $\langle \text{REQUEST}, req-id, op \rangle$ to a proposer in the current view. With the default assignment, every replica is a proposer. The identifier distinguishes requests for at-most-once execution; retries retain the same identifier. The receiving replica buffers the request in *cmds* and returns $\langle \text{REPLY}, req-id, result \rangle$ after executing it. The execution layer retains completed request identifiers and results, and answers a retry with the prior result instead of executing the operation again.

PROPOSE The fields are *view*, *log-slot*, *ack-slot*, and *cmds*. They identify the proposal's view and slot, the sender's cumulative acknowledgment, and the command payload.

PROPOSE-ACK The fields identify the current view and the sender's cumulative acknowledgment index *ack-slot*. The acknowledgment covers every slot through that index.

PROPOSE-NACK The fields are *view* and *nack-slot*; a sender may combine requests for multiple missing slots.

PROPOSE-RECOVER The fields are *view*, *recover-slot*, and *cmd*, where *cmd* retains the original proposal and its command view.

PROPOSE-NOOP The fields are *view* and *noop-slot*, denoting a no-op in a slot owned by the sender.

SKIP The fields are *view*, *slot-start*, and *slot-until*. The sender proposes no-ops for its own assigned slots in that inclusive range.

View-change messages A VIEW-CHANGE-REQUEST carries the new view and the candidate's last contiguous slot and its command view. A VIEW-CHANGE-VOTE identifies the new view and chosen candidate. The elected initiator proposes a VIEW-INIT command containing the new slot assignment, as specified in §B.8.

B.3 Normal Operation in Pulsing Mode

Each pulsing round has a publication phase and a consumption phase. A replica publishes at most one pulse per round; a pulse can carry a proposal, cumulative acknowledgment, and recovery or skipping records. The publication and consumption phases advance the proposal and acknowledgment stages of different log slots.

Publication phase. A proposer selects buffered requests, constructs a `PROPOSE` for *next-propose*, and includes its current *last-ack* as *ack-slot*. It records its own proposal locally, multicasts the pulse, and advances *next-propose*. A proposer with no client command can fill its assigned slot with a no-op, including through the skipping rule in §B.7. A replica without an assigned proposal slot publishes its acknowledgment and any pending recovery information without creating a new proposal.

Consumption phase. The replica consumes available pulses from earlier rounds while current pulses propagate. For a valid proposal from replica *r*, it records *cmds* at *log-slot* and updates *acked* [*r*] to the larger of its current value and *ack-slot*. A proposal beyond a hole remains buffered; it cannot advance the contiguous prefix. When a proposal fills the first hole, the replica scans the completed prefix to update *last-append* and, in an established view, *last-ack*. It also updates its own entry in *acked*. The next publication carries the resulting acknowledgment progress.

Commitment and execution. Let *c* be the *q*-th largest index in *acked*. A quorum has acknowledged every slot through *c*. In an established view, the replica advances *last-commit* through the smaller of *c* and its locally acknowledged prefix, never decreasing the committed index. The local bound prevents execution of commands that the quorum knows but this replica has not yet received. It executes newly committed commands in slot order, suppresses repeated execution of the same client request identifier, and returns results for requests it originally received. Before the new view's `VIEW-INIT` commits, the separate commitment barrier in §B.8 applies.

B.4 Responsive Mode

Responsive mode uses the same log and quorum rules but publishes messages as their triggering events occur. A proposer with a newly buffered request creates and multicasts a `PROPOSE`, records the proposal locally, and advances *next-propose*. On receiving a proposal for slot *s*, a replica records it and advances its contiguous prefix when possible. It multicasts `PROPOSE-ACK` only if *last-ack* $\geq s$; otherwise, the proposal remains behind an earlier hole. An outgoing message carrying the same cumulative acknowledgment can replace a separate `ACK` transmission. An `ACK` receiver monotonically updates the sender's entry in *acked* and applies the commitment rule.

Unused slots. An idle proposer must not leave assigned slots empty while other proposers advance. If a `NACK` requests

its own unused slot *s* $\geq$ *next-propose*, it proposes no-ops for its assigned slots from *next-propose* through *s* and advances the pointer past them. A `SKIP` can represent these no-ops compactly. Recovering a previously proposed command does not itself allocate a fresh local slot in responsive mode.

Mode transitions. A mode change changes when replicas transmit, while preserving the view, slot ownership, and meaning of acknowledgment evidence. A replica can consume either an explicit `ACK` or acknowledgment information carried by a pulse using the same state-update rule. The runtime coordinates transitions as described in §6.3. Neither transition permits a replica to acknowledge a missing proposal or bypass the view-change rules.

B.5 Proposal Recovery

A missing proposal prevents a replica from acknowledging the log beyond the first hole. A replica requests slot *s* with `PROPOSE-NACK`; in pulsing mode it attaches the request to its next pulse, and in responsive mode it transmits immediately. The `NACK` can accompany a new proposal and does not itself consume that proposal's slot.

Recovery eligibility. A replica can answer with `PROPOSE-RECOVER` only if it retains the requested proposal in its acknowledged prefix, so *s* $\leq$ *last-ack*. This restriction prevents a follower from distributing an unverified old-view suffix during reconciliation. The original proposer supplies missing current-view commands; other eligible replicas can also supply the same proposal. One recovery multicast can repair the same hole at several replicas. A receiver checks the view, records the recovered proposal, and advances the contiguous and acknowledged prefixes under the normal rule or the reconciliation rule, as appropriate.

Pulsing recovery. The recovery record occupies the next pulse's command payload instead of a fresh client command. A sender with an active assigned slot accounts for that slot with `PROPOSE-NOOP` and advances *next-propose*; a `SKIP` can encode the same no-op together with other unused slots. A sender without an assigned slot recovers the command without proposing a no-op. Thus, recovery does not abandon a newly consumed proposal slot. In responsive mode, the sender transmits the recovery immediately without this companion no-op.

B.6 Coordination and Failure Timers

Each proposer starts a coordination timer when it installs its slot assignment. The timer drives its locally proposed commands, including no-ops, toward commitment. At a timeout, it selects the earliest locally proposed slot *s* $>$ *last-commit*. If no such slot exists, it resets the timer without transmitting a repair. Otherwise, it checks the interval from *last-ack* + 1 through *s*. If the interval contains holes, it requests all missing proposals with `NACKs`. If *last-ack* $\geq s$, it retransmits its original proposal for *s* because another replica may have

missed it. Responsive mode transmits immediately; pulsing mode schedules the repair in a subsequent pulse. The proposer resets the timer after handling a timeout and whenever a local proposal commits.

A separate failure timeout detects lack of progress from a proposer and triggers a view-change attempt on suspicion of failure. Failure suspicion need not be accurate for safety. Candidates retry unsuccessful elections in higher views, using randomized backoff to resolve concurrent attempts. §B.8 gives the state transitions.

B.7 Skipping and Proposer Configuration

Eager skipping. Each replica tracks *latest-propose-slot*, the largest proposal slot it knows, and includes this progress hint in outgoing replica messages. A proposer uses received hints to identify unused assigned slots that precede other replicas' progress. It fills those slots with no-ops before its next transmission and represents a consecutive range of its assigned slots with `SKIP`. It advances *next-propose* beyond the range and never skips a slot for which it already proposed a different command. A receiver expands the range according to the sender's assignment and applies the normal prefix and commitment rules. In responsive mode, it acknowledges a completed skipped range when its cumulative prefix reaches the last skipped slot.

Proposer configuration. A view may assign slots to all replicas or only a subset. Changing the proposer set requires a view change; it does not change the quorum size for the fixed replica group. With one proposer, the remaining replicas still acknowledge its commands, so progress depends on a quorum rather than new proposals from every replica. The *R*-pulse, *R*-proposal accounting for the normal multi-proposer case assumes that all participating replicas have proposal slots; acknowledgment-only participants must also publish their evidence when the proposer set is smaller.

B.8 View Change

A replica starts a view change when it suspects that a proposer has failed or become unreachable. The new view selects an initiator whose log preserves committed commands and installs a slot assignment that excludes suspected failed proposers. Quorum intersection and log reconciliation preserve safety even when a suspicion is incorrect.

Starting an election. A candidate r' sets *view* to a higher number, *role* to candidate, and *voted-for* to itself. It clears buffered proposals beyond *last-append*, sets *next-propose* to *null*, resets peer entries in *acked* to zero, and sets *last-ack* to *last-commit*. It records its own acknowledged prefix in *acked*, clears *voted-by*, and inserts its own identifier. These updates suspend ordinary proposals and prevent acknowledgment evidence from different views from mixing. The candidate multicasts a `VIEW-CHANGE-REQUEST` containing the new view number, its *last-append*, and the command view at that slot.

Voting. A replica receiving a request for a higher view compares the candidate's contiguous log with its own. For logs ending at slots s and s' with command views v and v' , respectively, the first log is at least as up-to-date as the second exactly when

$$v > v' \quad \text{or} \quad (v = v' \text{ and } s \geq s').$$

If the candidate's log fails this check, the receiver starts an election in a still higher view. Otherwise, it adopts the requested view, becomes a follower, records the candidate in *voted-for*, and applies the same proposal-suspension and prefix-reset operations as the candidate. It replies to that candidate with `VIEW-CHANGE-VOTE` and does not vote for another candidate in the same view. The candidate counts distinct votes for itself in the current view in *voted-by*. Once this set reaches q replicas, it becomes the initiator.

Proposing the new assignment. The initiator sets *view-base* to one more than the last contiguous slot in the log on which it won the election. It proposes `VIEW-INIT` at *view-base*, retaining all earlier commands and their original command views. This command defines the proposer set and slot assignment beginning at *view-base* + 1. The initiator acknowledges its complete prefix through *view-base* but leaves *next-propose* as *null* until `VIEW-INIT` commits locally. Thus, it cannot create ordinary proposals under an assignment that has not yet committed.

Adopting the initiator. A replica learning the elected initiator's proposal in a higher view adopts that view and becomes its follower. A candidate or follower learning the elected initiator in its current view adopts that initiator, even if it voted for a different candidate. It suspends ordinary proposals, discards buffered entries beyond its contiguous prefix, and resets acknowledgment evidence to its committed prefix as above. A replica already following that initiator in the same view does not repeat these resets for subsequent messages. On first receiving this initiator's `VIEW-INIT`, the follower sets *view-base* to its slot and removes any uncommitted suffix starting there. It retains the new command separately until it reconciles the preceding prefix. Duplicate copies of the same `VIEW-INIT` do not repeat truncation or reset acknowledgment state, including after the command commits. It retains its earlier contiguous commands but cannot acknowledge an uncommitted inherited suffix merely because its log contains those commands.

Reconciling the prefix. The follower requests proposals beyond *last-ack* through *view-base* - 1 using `PROPOSE-NACK`. A recovery responder must have acknowledged the requested slot. For a recovered proposal at slot s with $\text{last-ack} < s \leq \text{last-append}$, the follower compares its retained command view with the recovered command view. If they match, it advances *last-ack* through s ; the log-matching property in Claim 10 establishes agreement on that prefix. If they differ, it removes its uncommitted suffix starting at s , reducing *last-append*

to $s - 1$, and requests the missing suffix again. Once its acknowledged prefix reaches its appended prefix, it can append further recovered proposals and advance both indices. After reconciling all slots below *view-base*, it appends the pending VIEW-INIT and acknowledges that slot. The new command cannot itself serve as evidence that the inherited prefix matches the initiator. Recovery preserves each proposal's original command view, even though the enclosing message carries the new view. A follower publishes its updated acknowledgment so that the initiator and peers can observe reconciliation progress.

Committing the view. Let c be the q -th largest current-view acknowledgment index. A replica uses this quorum evidence to advance commitment only when $c \geq \text{view-base}$: the quorum must acknowledge VIEW-INIT and its entire preceding prefix. The replica must also have locally reconciled that prefix through *view-base*. It then commits VIEW-INIT together with earlier uncommitted commands, executes newly committed client commands in order, and installs the new assignment. Its *next-propose* becomes its first assigned ordinary slot, or remains *null* if it has no assigned slots. Subsequent commitment follows the normal quorum and local-prefix bounds. This guard prevents new-view acknowledgments for an incomplete inherited prefix from committing commands before the replica establishes the new view.

Retrying and rejecting stale messages. A candidate that cannot obtain a quorum before its election timeout retries in a higher view with randomized backoff. Replicas ignore lower-view messages throughout election, reconciliation, and normal operation. They continue recovery requests while reconciliation is incomplete, and can initiate another view change if the selected initiator stops making progress.

B.9 Safety Proof

We consider the problem of replicating a log across $N = 2f + 1$ replicas using the CHORA protocol specified in Appendix B. Each replica has a local log. The log consists of an infinite series of *slots*, starting from index 1. Replicas propose *commands* for the log slots. Each log slot can be either empty, or contain one unique command. We present the formal proof of the following theorem:

Theorem 1. Safety: If a command c is committed at slot s , no other command can be committed at s .

Let's first clarify some conventions for ease of illustration. When *any log* is referred to, we are discussing all possible local logs of any replica at any given time. For a certain log, we denote the command at slot s using $s.\text{cmd}$ and. For a certain command c , we denote its view as $c.\text{view}$. We say that a command at slot s is an *appended command* if $s \neq \text{append-slot}$. When a command is *appended* to a log at slot s , we denote the cases where the command is added to the log at s , and it becomes an appended command after the

addition. When a command is *committed*, it denotes that the command is locally committed in some log. Also, we assume that there is a *null* command at slot 0 of all logs, and the command's view is 0.

In CHORA, there is a basic requirement for all replicas, which is trivial because no malicious node is considered:

Fact 1. If a replica r proposes a command c at slot s in view v , it never proposes a different command at s in v .

Also, from the protocol, we can notice that:

Fact 2. An initiator only adds commands from the current view to its log.

Fact 3. A command can and only can be added to a log in two cases:

1. A replica learns a proposal from the same view with a PROPOSE OR a PROPOSE-RECOVER.
2. A replica learns the command with a PROPOSE-RECOVER from an initiator.

Claim 1. If command c is added to any log in view v , then c has been proposed and $c.\text{view} \leq v$.

Proof. Without loss of generality, assume v is the smallest view that c is added. Let's discuss the two possible cases in Fact 3.

In the first case c is proposed in the current view ($c.\text{view} = v$).

In the second case, given Fact 2, c is added before view v . This violates the assumption that v is the smallest view that c is added to a log. $\square$

Claim 2. If a replica r is elected as an initiator in view v , then no other replica can be elected as an initiator of v .

Proof. If another replica r' is elected as an initiator in v , both r and r' have received at least $f + 1$ VIEW-CHANGE-VOTE s from different replicas. Due to quorum intersection, there exists at least one replica r'' that has sent VIEW-CHANGE-VOTE to both r and r' . This violates the protocol where a follower ignores a VIEW-CHANGE-REQUEST from any other candidate in the same view after updating *voted-for* before replying VIEW-CHANGE-VOTE. $\square$

Claim 3. If a INIT-VIEW for view v exists in any log, then no other INIT-VIEW for v exists in any log.

Proof. If the target INIT-VIEW exists in a log, it must have been added to the log and thus must have been proposed (Claim 1). In a view, only an initiator proposes a INIT-VIEW. Given Fact 1 and Claim 2, the proposed INIT-VIEW is unique. $\square$

To simplify the following illustration, let's introduce more conventions at this point. If a replica is elected in the view, we call it *the initiator* of the view and denote it with $v.\text{initiator}$. Correspondingly, we use *the base log* of v to denote the local log of the initiator when it starts election in v , if the initiator

exists. Also, we use $v.init$ to denote the unique INIT-VIEW of v if it exists.

Claim 4. If command c is proposed for slot s in view v , then no other command can be proposed for s in v .

Proof. Replicas propose in v following $v.init$. Given Claim 3 and Fact 1, we know that every proposal is unique. $\square$

Claim 5. If command c is appended at slot s following c' , then c is appended at slot s following c' in $v = c.cmd$ for some log.

Proof. Without loss of generality, let's discuss the smallest view v' where this appending happens. Given Claim 1, we have $v' \geq v$. Let's discuss the two cases in Fact 3 separately.

For the first case, the view is v ($v' = v$) when the appending happens.

For the second case, the checking logic of the follower before appending ensures that the append only happens if the command in the previous slot is consistent with the initiator's log. If the claim doesn't hold, then we know c is not appended after c' to the initiator's log in any view $v'' < v'$. Also, we know that c is not appended to c' in v' given Fact 2. So, the initiator's log doesn't contain c at s following c' . This leads to a contradiction.

With this serving as an induction step, we can know that if the targeted append doesn't happen in view v , it cannot happen in any view bigger than v . $\square$

Claim 6. For any log, if $s < s' \leq last-append$, then $s.cmd.view \leq s'.cmd.view$.

Proof. From Claim 5, it is known that $s'.cmd$ is appended after $s.cmd$ in $s'.cmd.view$ for some log. Assume that $s.cmd.view = v$ and $s'.cmd.view = v'$. Assume that for this log, $s.cmd$ is appended in view v'' . From Claim 1 we have $v \leq v''$. The fact that $s.cmd$ exists when $s'.cmd$ is appended denotes that $v'' \leq v'$. So, we have $v \leq v'$. $\square$

Claim 6 also implies the following statement:

Claim 7. In a log, if $s < s' \leq append-slot$ and $s.cmd.view = s'.cmd.view = v$, then for any s'' that satisfies $s < s'' < s'$, $s''.cmd.view = v$. In other words, the commands with the same view are consecutive in a log.

Claim 8. For any log, if slot s is the smallest appended slot in view v that satisfies $s.cmd.view = v$, then $s.cmd$ is $v.init$.

Proof. From the protocol, we can see that a replica only appends other commands from the current view after the current view's VIEW-INIT has been appended. Also, a replica never removes any command from the current view. So, $s.cmd = v.init$. $\square$

Claim 9. If an appended command c exists at slot s in some log, then for any log, if an appended command c' exists at slot s and $c.view = c'.view$, then $c = c'$. Also, c and c' are appended following the same command.

Proof. Assume $c.view = c'.view = v$. According to Claim 5, both appendings happen in v . According to Claim 4, $c = c'$.

Assume c is appended following c_p . If $c_p.view = v$, then according to the analysis above, it is unique. Otherwise, because of Claim 7 and Claim 8, $c = v.init$. According to the protocol, the check of followers when appending $v.init$ in v ensures that c_p is unique and is consistent with the initiator's log. Given Claim 5, c_p is unique. $\square$

Claim 10. Log Matching Property: If two logs contain an identical appended command with the same slot s and view v , then the two logs are identical up to slot s .

Proof. If the claim doesn't hold, without loss of generality, assume s' is the biggest slot that violates the claim. In other words, $s' \leq s$ and the commands are not identical. Then the commands at $s' + 1$ are identical (c) and are appended. When looking at c , Claim 9 is violated because it is appended to two different commands. $\square$

Claim 11. Assume the last appended slot of l and l' are s and s' respectively, if $s \geq s'$ and $s.view = s'.view = v$, then the appended commands of the two logs (c and c') at s' are identical.

Proof. If $c.view = c'.view$, according to Claim 9, we have $c = c'$. Otherwise, given Claim 7 and Claim 8, we know that for l , $v.init$ is in a slot bigger than s' , while for l' , $v.init$ is in a slot no bigger than s' . This violates Claim 9. $\square$

Claim 12. Log Inclusion Property: Assume the last appended slot of l and l' are s and s' respectively, if $s \geq s'$ and $s.view = s'.view = v$, then the commands at a slot no bigger than s' are identical in the two logs for any $s'' \leq s'$. In other words, l includes all appended commands in l' .

Proof. According to Claim 11, we know that the appended commands at s' are identical. Given Claim 10, we know that the two logs are identical up to slot s' . $\square$

Claim 13. If c is committed in view v and $c.view = v$, then c is included in the log of the initiator of any view v' that satisfies $v' > v$.

Proof. If the claim doesn't hold, without loss of generality, assume v' is the smallest view that violates the claim. Since c is committed in view v , so at least $f + 1$ replicas append c in v (given Claim 1, c can not be appended in a view smaller than v). Also, at least $f + 1$ replicas vote for $v'.initiator$ in v' . So, there exists one replica r that both appends c in v and votes for $v'.initiator$ in v' . Also, according to the protocol, a replica only removes a command when it is inconsistent with the initiator's log, and the leaders for views between v and v' all include c in the log (since v' is the smallest view that doesn't satisfy the claim). So, c is still included in the log of r when it votes for $v'.initiator$.

Assume the last appended command in the log of r is c_1 when it votes for $v'.initiator$. Also, assume the last appended

command in the base log of v' is c_2 . According to the protocol, there are two possible cases.

In the first case, $c_1.view = c_2.view$ and the base log of v' has a bigger *append-slot*. In this case, we know that c is included in the base log according to Claim 12.

In the second case, $c_1.view < c_2.view$. Let's denote $c_2.view$ as v'' , so $v'' < v'$. According to Claim 6, $v = c.view \leq c_1.view$. So, $v \leq v'' < v'$. Also, $v''.init$ exists in the base log of v' according to Claim 8. Given Claim 10, the base log of v' and the base log of v'' are at least identical up to $v''.init$. Given that v' is the smallest view that violates the claim, the base log of v'' includes c . So, the base log of v' also includes c . This leads to a contradiction given the definition of the base log. $\square$

Claim 14. *Initiator Completeness Property:* If c is committed in v , then c is included in the log of the initiator of any view v' that satisfies $v' > v$.

Proof. Claim 13 proves the case when $c.view = v$. For $c.view \neq v$, according to the protocol, the commit happens when $v.init$ is committed in v . Also, we know that $c.view < v$ from Claim 1. So, for $v.init$, we know that it exists in the log of the initiator of any view bigger than v , which includes v' . According to Claim 10, we know that c is also included in the log of the initiator of v' . $\square$

Claim 15. If command c is committed at slot s in view v , for any v' that satisfies $v' \geq v$, any command c' that is committed at s in v' is c .

Proof. If $v = v'$, according to Claim 9, $c' = c$. Otherwise, c' is committed when $v'.init$ is committed, and c' is included in the base log of v' . According to Claim 14 and Claim 9, $c' = c$. $\square$

Using induction, Claim 15 leads to Theorem 1.